

Exercise 3

Model 1 : DeepSeek-R1 (DeepThink Mode)

Using generative AI as a university student is a game-changer—if you use it as a **personal, patient TA** rather than a cheating machine. The goal is to build *intuition* and *debug your thinking*, not to skip the work.

Here is a practical, subject-specific playbook for Probability & Statistics, DBMS, and Software Engineering, along with the **golden rules** of prompting.



The 3 Golden Prompts That Work for ALL Subjects

Before we dive in, memorize these prompt structures:

1. **The Feynman Technique:** *"Explain [concept] to me as if I'm 15, then give me a real-world analogy."*
2. **The Socratic Tutor:** *"Don't give me the answer yet. Ask me guiding questions to help me figure out [problem] on my own."*
3. **The Quiz Master:** *"Give me a 5-question pop quiz on [topic]. Grade my answers, tell me where I'm wrong, and don't move on until I get it right."*

1. Probability & Statistics (The Math Heavyweight)

The biggest struggle here is *intuition* (why does this formula exist?) and *calculation errors*.

- **Visualize the Abstract:** Ask, *"Describe the Central Limit Theorem using a metaphor of brewing multiple pots of tea."* Or, *"Write a Python script using Matplotlib to visually simulate rolling a die 10,000 times to show the Law of Large Numbers."* (Seeing it happen cements the math).
- **Walk Through Derivations:** Don't ask for the final answer. Ask: *"Walk me through the steps of deriving the Maximum Likelihood Estimate for a Poisson distribution. Show the math, but explain every algebraic step in plain English."*
- **Generate Custom Practice Data:** Ask, *"Generate a CSV dataset with 50 rows representing exam scores and study hours. Give me the data, but do NOT give me the correlation coefficient. Let me calculate it myself, and then I'll paste my answer for you to check."*
- **Debug Your Process:** Paste your worked-out solution and say, *"I got a p-value of 0.04, but my friend got 0.06. Check my logic line-by-line and tell me where I misinterpreted the null hypothesis."*



2. Database Management Systems (DBMS - The Logic & Design Heavyweight)

This subject is about set-theory (SQL), design (Normalization), and internals (Indexing/Transactions).

- **Master Normalization (1NF, 2NF, 3NF, BCNF):** Give it a messy, denormalized table (e.g., OrderID, CustomerName, ProductID, ProductPrice, Quantity).
Prompt: *"Act as a strict database professor. Walk me through normalizing this table to 3NF. Do it step-by-step: first identify the functional dependencies, then split the tables, and explain why each split removes a specific anomaly."*
- **Practice Complex SQL without a Server:** Prompt: *"Give me a schema for a Library Management System (Books, Members, Loans). Write 5 tricky SQL queries*

I must write—including nested queries and JOINS with GROUP BY. After I write my SQL, critique my syntax, efficiency, and tell me if an index would speed it up."

- **Understand Internals (B-Trees / Transactions):** Ask for interactive storytelling: *"Explain the ACID properties using a bank transfer between two friends. Then, explain how a B-tree index actually retrieves a specific 'where user_id = 1024' record by walking me through the tree branches visually using text characters."*
 - **Design ER Diagrams:** Describe your university's course registration process and ask: *"Based on this description, give me a list of entities, attributes, and cardinalities. I will draw the ER diagram myself, then describe it back to you so you can check if my relationships are correct."*
-

3. Software Engineering (The Process & Architecture Heavyweight)

This isn't just coding; it's about design patterns, testing, SDLC, and system design.

- **Design Patterns Made Real:** Don't just memorize definitions. Ask: *"Give me a real-world coding scenario (in Python/Java) for a pizza restaurant. Show me the 'bad' way to write it (spaghetti code), then refactor it using the Factory Pattern, and explain why the refactored version is better for future maintenance."*
- **Code Reviews (The Ultimate Learning Hack):** You *must* write your own code for assignments, but use AI as a Senior Dev. Paste your function and say: *"Act as a senior engineer. Do NOT rewrite my code yet. First, give me a code review highlighting 3 SOLID principle violations, 2 potential security flaws, and 1*

performance bottleneck. Then give me hints on how to fix them, but let me write the fixes."

- **UML and Architecture:** Ask: *"Describe the architecture for a ride-sharing app. Convert your description into PlantUML code so I can paste it into a viewer and see a Class Diagram. Explain why you chose Composition over Inheritance for the 'Vehicle' class."*
 - **Simulate Agile/Scrum:** If you have a group project, paste your project requirements and ask: *"Help me write 10 detailed User Stories using the 'As a... I want... So that...' format. Then, prioritize them into 2 Sprints and explain which stories depend on each other."*
-

⚠ The Critical Warnings (Read This!)

Danger

How to Avoid It

**Hallucinated
Math/Formulas**

LLMs are bad at exact arithmetic. **Never** ask it to calculate a complex p-value standard deviation purely by text. Instead, ask it to *write the Python/R code to* the calculation, and run that code yourself.

**Outdated SQL Syntax /
DB specifics**

DBMS systems (MySQL vs. PostgreSQL) differ. Always specify: *"Write this for PostgreSQL/MySQL version X"* and verify against your textbook.

**Stealing Architecture
(Academic Misconduct)**

Never say: *"Write my full 300-line software engineering project."* You'll learn nothing. **Always** say: *"Give me the structure/infrastructure/starter boilerplate,"* and write the core business logic yourself.

Danger

How to Avoid It

Trusting the AI blindly

If the AI explains a Software Engineering concept (like Agile) differently than your professor's slides, **your professor is always right for the exam**. Use AI to supplement, but revise using your course materials first.

One final pro-tip: Create a "Personal Tutor" custom instruction in your AI settings (if using ChatGPT/Claude) that says: *"Never give me the full answer immediately. Always start by asking me what I already know about the topic, then give hints, and only reveal the solution after I try."* This forces active recall, which is how you actually learn!

Model 2: Qwen3.7-Plus (Thinking Mode)

Using generative AI as a university student can be transformative if you treat it as a ****personalized, 24/7 tutor**** rather than an answer generator. The key is to use it to deepen your understanding, test your knowledge, and bridge gaps in your learning.

Here is a practical, subject-by-subject guide on how to leverage generative AI for Probability & Statistics, Database Management Systems (DBMS), and Software Engineering, along with general best practices.

🎲 1. Probability and Statistics

This subject is highly conceptual and math-heavy. AI excels at breaking down complex formulas into intuitive explanations and generating endless practice problems.

* **Build Intuition, Not Just Memorization**: Ask the AI to explain the *why* behind a formula.

* **Prompt**: "Explain the intuition behind Bayes' Theorem using a real-world medical testing example, without using heavy math jargon first. Then, show me the formula."

* **Step-by-Step Problem Solving**: When stuck on a homework problem, don't ask for the final answer. Ask for a hint or the first step.

* **Prompt**: "I'm trying to solve this problem: [insert problem]. I know I need to use a binomial distribution, but I'm stuck on setting up the parameters. Can you give me a hint without giving away the full solution?"

* **Generate Practice Quizzes**: Test your active recall.

* **Prompt**: "Generate 5 multiple-choice questions on hypothesis testing (specifically Type I and Type II errors). Provide the answers and detailed explanations only after I attempt them."

* **Translate Math to Code**: Bridge the gap between theory and application.

* **Prompt**: "Show me how to simulate the Central Limit Theorem using Python and `matplotlib`. Comment each line of code to explain what statistical concept it represents."

📖 2. Database Management Systems (DBMS)

DBMS combines theoretical concepts (normalization, ACID properties, indexing) with practical skills (SQL, schema design). AI is excellent at acting as a sounding board for your designs.

* ****Normalize Schemas Interactively****:

* ***Prompt***: "Here is my unnormalized table for a university course registration system: [insert table]. Walk me through the steps to convert this to 1NF, 2NF, and 3NF. Ask me questions along the way to check my understanding."

* ****Debug and Optimize SQL****: Instead of just asking the AI to write a query, write it yourself first and ask the AI to review it.

* ***Prompt***: "Here is my SQL query to find the top 3 students by GPA per department: [insert query]. It's running slowly. Can you explain how a database might execute this, and suggest an index or rewrite to optimize it?"

* ****Generate Mock Data****: Great for testing your own applications or queries.

* ***Prompt***: "Generate 10 rows of realistic mock data in SQL `INSERT` format for a `Books` table with columns: `book_id`, `title`, `author`, `publish_year`, and `genre`."

* ****Explain Execution Plans****:

* ***Prompt***: "Explain what a B-Tree index is, how it speeds up a `WHERE` clause, and under what circumstances it might actually slow down a database."

🏗️ 3. Software Engineering

SE is about processes, architecture, design patterns, and writing maintainable code. AI can simulate senior engineers, review your designs, and clarify abstract concepts.

* ****Learn Design Patterns with Context****:

* *Prompt*: "Explain the Observer design pattern. First, give me a real-world analogy (like a YouTube subscription). Then, show me a clean, commented implementation in Java/Python."

* **Simulate Code Reviews**: Paste your code and ask for constructive, professional feedback.

* *Prompt*: "Act as a senior software engineer. Review this code for adherence to SOLID principles, readability, and potential edge cases. Don't rewrite it for me; just point out the issues and explain why they matter."

* **Generate UML and Architecture Diagrams**: Use AI to generate Mermaid.js code, which you can render into diagrams.

* *Prompt*: "Write Mermaid.js code for a sequence diagram showing the interaction between a User, a Web Server, an Authentication Service, and a Database during a login process."

* **Understand SDLC and Agile**:

* *Prompt*: "Compare Waterfall and Agile methodologies. Create a table showing the pros, cons, and best-use cases for each in a university capstone project context."

🧠 Universal AI Study Strategies

1. **The Feynman Technique**: Explain a concept to the AI and ask it to grade your understanding.

* **Prompt**: "I'm going to explain how a B+ Tree works. Here is my explanation: [your text]. Rate my explanation out of 10, point out any inaccuracies, and tell me what I missed."

2. **The Socratic Method**: Force the AI to teach you by asking questions.

* **Prompt**: "I want to learn about Deadlocks in DBMS. Act as a Socratic tutor. Ask me one guiding question at a time to help me discover the four necessary conditions for a deadlock. Do not give me the answers."

3. **Summarize Lecture Notes**: Paste your messy, rapid-fire lecture notes and ask the AI to structure them.

* **Prompt**: "Organize these raw notes into a structured study guide with clear headings, bullet points, and a 'Key Takeaways' section."

⚠️ Crucial Caveats & Best Practices

- * **Beware of Hallucinations**: AI can confidently state false information, especially with complex math or niche database edge cases. **Always verify** critical formulas, syntax, or concepts with your textbook, lecture slides, or official documentation.
- * **Maintain Academic Integrity**: Use AI to *learn how to do the work*, not to *do the work for you*. Submitting AI-generated code or essays as your own is usually a violation of university policy and robs you of the actual learning.
- * **Provide Context**: The better your prompt, the better the output. Mention your current level of understanding (e.g., "I am a sophomore computer science student") so the AI calibrates its explanation.
- * **Don't Lose the "Struggle"**: Productive struggle is where actual learning happens. If you immediately ask AI for the solution to every hard problem, you will fail exams when the AI isn't there. Use it to get unstuck, not to bypass the thinking process.

****Pro Tip**:** Create a dedicated "Study Bot" persona prompt at the start of a chat: **"You are an expert, patient university tutor in Computer Science and Mathematics. Your goal is to help me understand concepts deeply using analogies, step-by-step guidance, and practice questions. Never give me the direct answer to a homework problem; instead, guide me to find it myself."**

Model 3: Grok 4.5: (DeepSearch)

****Generative AI tools (ChatGPT, Claude, Grok, Gemini, specialized coding assistants, etc.) work best as interactive tutors, practice generators, explainers, and feedback providers****—not as substitutes for lectures, textbooks, problem sets, or your own reasoning. Use them actively: ask follow-up questions, request alternative explanations, verify outputs against reliable sources, and always attempt problems yourself first.

General Effective Practices

- ****Prompt for depth**:** “Explain [concept] like I’m a second-year undergrad who knows basic calculus/set theory. Give an intuitive analogy, a formal definition, a worked example, common pitfalls, and 2–3 practice questions with solutions.”

- **Socratic mode**: Ask the AI to quiz you or guide you step-by-step without giving the full answer immediately.
- **Iterative learning**: Paste your notes or a confusing lecture slide and request clarification, reorganization, or connections to prior topics.
- **Verification habit**: Cross-check formulas, theorems, SQL syntax, or design principles against textbooks, official docs, or course materials. AI can hallucinate details or outdated practices.
- **Study aids**: Generate flashcards, concept maps (described in text or Mermaid), summary sheets, or spaced-repetition style quizzes.
- **Ethical use**: Use AI to understand and practice, not to submit generated assignments or code as your own. Follow your university's academic integrity policies.
- **Tools**: General chat models for concepts; coding-focused ones (or those with code interpreters) for DBMS and software engineering; multimodal models for diagram descriptions or visual explanations.

Probability and Statistics

Focus on intuition + formal understanding + computation.

- Request layered explanations: intuitive story → mathematical definition → derivation sketch → real-world application (e.g., “Explain Bayes’ theorem with a medical testing example, then the general formula, then how it appears in machine learning”).
- Generate and solve problems: “Create 5 problems on conditional probability / discrete distributions / hypothesis testing at intermediate difficulty, provide solutions after I attempt them.”
- Explore edge cases and misconceptions: “What are common student mistakes with the central limit theorem or p-values?”
- Simulation help: Ask for Python (NumPy/SciPy/pandas) or R code to simulate distributions, Monte Carlo methods, or visualize sampling distributions, then modify and run it yourself.
- Connections: Link topics (e.g., how variance relates to confidence intervals, or likelihood in both stats and later ML).
- Visuals: Request step-by-step descriptions of density plots, QQ-plots, or decision trees for hypothesis tests that you can sketch or plot.

Practice deriving results yourself after seeing an AI walkthrough, and use real datasets from your course or public sources to apply concepts.

Database Management Systems (DBMS)

Emphasize concepts, design, and hands-on querying/implementation.

- Concept clarification: “Compare relational vs. NoSQL models; explain normalization (1NF–BCNF) with a progressive example starting from a denormalized table; discuss ACID properties with failure scenarios.”
- Schema and query practice: Describe a domain (e.g., university enrollment) and ask for ER diagrams (text or Mermaid), relational schemas, then progressive SQL exercises (SELECT → joins → aggregates → subqueries → window functions → transactions).
- Optimization and internals: “Explain indexing (B+ trees), query execution plans, and when a full table scan occurs. Give example EXPLAIN outputs.”
- Hands-on coding: Generate sample schemas + data, then practice writing queries. Ask the AI to review *your* SQL for correctness, efficiency, and style. Use local tools (SQLite, PostgreSQL, MySQL) or cloud sandboxes to execute.
- Advanced topics: Transaction isolation levels with concurrent scenario examples; concurrency control (locking, MVCC); NoSQL query patterns if covered in your course.

- Design critique: Submit your own schema or query and request feedback on normalization, integrity constraints, or performance.

Always test queries and designs yourself. AI is excellent for generating varied practice data and alternative query formulations.

Software Engineering

Cover processes, design, coding practices, and project skills.

- Process and principles: “Explain Agile vs. Waterfall with trade-offs for a student project; describe SOLID principles with before/after code examples in [language]; walk through requirements elicitation, use cases, and user stories.”
- Design and architecture: Request UML-style descriptions (class, sequence, component diagrams in text/Mermaid), design pattern explanations (Factory, Observer, etc.) with concrete code, and trade-offs (e.g., microservices vs. monolith for a course project).

- Coding and quality: Paste your code or describe a feature and ask for refactoring suggestions, test cases (unit/integration), edge-case identification, or explanations of design decisions. Generate boilerplate or skeleton code as a starting point, then implement the logic yourself.
- Testing and DevOps basics: “Create a test plan for [module]; explain CI/CD pipeline stages; show example Git branching strategies and commit messages.”
- Project support: Help break down a large assignment into milestones, suggest documentation structure, or review architecture decisions. For team projects, use it to clarify roles or generate meeting agendas/checklists.
- Common pitfalls: Ask about technical debt, code smells, estimation techniques, or risk management in student-scale projects.

Pair AI suggestions with actual implementation, version control practice, and peer/code-review feedback. Use it to explore multiple design alternatives quickly.

Practical Workflow Tips

1. After lectures, feed key points or confusing sections to the AI for reinforcement and expansion.

2. Attempt homework/problems first; use AI for hints, alternative solutions, or debugging after your attempt.
3. Build a personal prompt library for recurring needs (e.g., “SQL practice generator for joins”).
4. Combine with other resources: textbooks for rigor, official documentation (SQL standards, language docs), visualization tools, and IDEs.
5. Track learning: Periodically ask the AI to quiz you comprehensively or generate a self-assessment on a topic.
6. Time management: Use AI to create realistic study schedules or prioritize weak areas based on your self-reported understanding.

Generative AI accelerates feedback loops and provides unlimited patient examples, which is especially powerful for these subjects that mix theory, formal methods, and practical application. The biggest gains come from treating it as a collaborator that forces you to articulate questions and evaluate answers critically. Consistent deliberate practice with verification will deepen understanding far more than passive generation of explanations.

Key Findings

- **Active Learning:** Using AI as a tutor and getting step-by-step guidance helps students understand concepts better than simply receiving answers.
- **Visual Learning:** AI can generate Python scripts to create graphs, making statistics easier to understand.
- **DBMS Support:** AI helps simplify database normalization (1NF–3NF) and SQL query writing.
- **Problem Solving:** AI can help students understand the logical steps used to solve and debug technical problems.

Quick Tool Comparison

- **DeepSeek-R1 (DeepThink):** Best for mathematics, logic, and complex problem-solving.
- **Qwen (Thinking):** Best for structured explanations and study guides.
- **Grok (DeepSearch):** Best for current web research and industry trends.

Best Tool

Winner: DeepSeek-R1 (DeepThink)

Reason: It is useful for university-level mathematics, databases, and software engineering because it focuses on logical problem-solving and helps students understand how solutions are developed.